Online Assessment 1

Scaler School of Technology | ML System Design and LLMOps | Week 1, Session 1 – Week 3, Session 2 | 15 Marks

Assessment Overview

Total Marks	15	**Duration**	60 minutes
Question Types	Descriptive + Multi-MCQ + Match		

Assessment Structure

Section	Type	Questions	Marks	Time (approx.)
A	Descriptive Questions (sequential sub-parts)	4	12	∼40 min
B	Multiple Options Correct MCQ	4	2	∼12 min
C	Match the Following	1	1	∼8 min
Total		**9**	**15**	**60 min**

Marking Scheme

Section A Descriptive

Show all formulae and intermediate calculations. A correct final number with no working receives at most half credit. Sub-parts build on each other — an error carried forward from an earlier sub-part is not re-penalized in later sub-parts, provided your method is consistent (method marks apply).

Section B Multiple Options Correct MCQ

Each question may have more than one correct option. No credit is given for a partially correct selection — you must select exactly the full correct set. No negative marking.

Section C Match the Following

No credit is given for guessing. Several Column A items map to more than one Column B item; each Column B item may be used once, more than once, or not at all; and some Column B items are unused decoys, including a deliberate near-miss.

Exam Administration Notes

- This paper has three sections: Descriptive (12 marks), Multiple Correct MCQs (2 marks), Match the Following (1 mark)
- Show all formulae and intermediate calculations in the descriptive section
- MCQs may have more than one correct option — select the full correct set for credit
- Match the Following: several Column A items map to more than one Column B item; some Column B items are unused decoys (one is a deliberate near-miss)
- All answers typed using the keyboard (ASCII only)
- Browser lockdown and proctoring enabled

Instructor Note (omit from student copy)

Descriptive Q1 and Q2 (7 marks total) are rooted entirely in Week 3, Session 1 (LLM Serving and Inference Optimization). Combined with partial coverage inside MCQ/Match sections, Week 3 Session 1 content accounts for ≈50% of total marks.

Section A — Descriptive Questions (12 marks)

Show all formulae and intermediate calculations. A correct final number with no working receives at most half credit. Method marks apply.

Q1 Descriptive | Memory Systems Under Load — LLaMA-2-13B on a Shared Cluster | 3.5 marks

Your team is deploying **LLaMA-2-13B** for an internal document-QA tool. The model has **40 transformer layers**, **40 attention heads**, and a **head dimension of 128**. You are given a cluster of **2× A100 GPUs, 80 GB VRAM each (160 GB total)**. Assume fp16 (2 bytes/value) unless a sub-part says otherwise.

(a) Calculate the VRAM required to load the LLaMA-2-13B weights at fp16. Show the formula. **[0.5 marks]**

(b) Each incoming query uses a **3072-token** context (prompt + generated tokens combined). Using the KV cache formula KV mem $= 2 \times L \times H \times d \times S \times$ bytes, calculate the KV cache memory required **per request** at fp16. **[0.75 marks]**

(c) After loading the weights (from part a), how many **concurrent requests** can the cluster serve before KV cache alone exhausts the remaining VRAM? Assume no other overhead (activations, framework, etc.) for this calculation. **[0.75 marks]**

(d) Your product team demands the cluster support **3× more concurrent requests** than your answer in part (c), without adding GPUs. You may apply **exactly one** of: (i) switch weights to INT4 while keeping KV cache at fp16, or (ii) apply PagedAttention while keeping everything else unchanged. Using numbers, determine whether **either single option alone** is sufficient to hit the 3× target. State your assumption about how much fragmentation PagedAttention removes. **[0.75 marks]**

(e) Explain, in terms of the **roofline model** (compute-bound vs. memory-bandwidth-bound), *why* reducing weight precision (option i) helps generation throughput more than one might naively expect from a "memory savings" argument alone. Reference what happens at each decoding step, not just total footprint. **[0.75 marks]**

Expected Answer

(a) LLaMA-2-13B has 13B parameters. At fp16 (2 bytes/param): $13\text{B} \times 2\,\text{bytes} = \mathbf{26}$ GB.

(b) $L = 40$, $H = 40$, $d = 128$, $S = 3072$, bytes $= 2$. KV mem $= 2 \times 40 \times 40 \times 128 \times 3072 \times 2 = 3{,}019{,}898{,}880$ bytes $\approx \mathbf{2.81}$ GB per request (accept 2.7–2.85 GB depending on 2^{30} vs. 10^9 rounding).

(c) Remaining VRAM: $160 - 26 = 134$ GB. Concurrent requests: $\lfloor 134/2.81 \rfloor = \mathbf{47}$ (accept 46–48).

(d) Target: $3 \times 47 \approx 141$. *Option (i) INT4 weights:* weights drop to $26/4 = 6.5$ GB, remaining VRAM $= 153.5$ GB, KV cache unchanged at $\sim$2.81 GB/req $\Rightarrow \lfloor 153.5/2.81 \rfloor \approx 54$ requests — far short of 141, since KV cache (not weights) is now the binding constraint. *Option (ii) PagedAttention:* assuming naive allocation wastes up to $\sim$90% of KV cache, recovering 2–3× effective capacity yields roughly 94–141 requests — close to or at target, but sensitive to the assumed fragmentation rate. **Neither option alone is guaranteed sufficient**; PagedAttention gets far closer because KV cache is the dominant memory consumer per unit of added capacity at this scale.

(e) Generating each token requires reading all weights from HBM at least once — a fixed memory-bandwidth cost paid *per decoding step*, independent of useful compute performed. Reducing precision (fp16→INT4) cuts bytes streamed per step, directly attacking the dominant bottleneck (bandwidth, not FLOPs). Since LLM inference is memory-bandwidth bound, a 4× byte reduction roughly translates to a proportional reduction in the memory-bound portion of each step's cost — which is most of the step. This is why quantization helps more than a naive "smaller model" argument suggests.

Criteria	Marks
(a) Correct weight VRAM with formula shown	0.5
(b) Correct KV cache formula and calculation	0.75
(c) Correct concurrent-request count using (a)+(b)	0.75
(d) Numerically evaluates both options and identifies KV cache as bottleneck	0.75
(e) Correct roofline / memory-bandwidth reasoning at the per-step level	0.75

Q2 Descriptive | From Draft Models to Dollars — A Serving Stack Redesign | 3.5 marks

A company serves a code-completion product using a **70B parameter model** at fp16 on a static-batching inference stack. GPU utilization: **24%**. Throughput: ∼5 requests/second on their 4×A100 cluster. They are considering three independent changes: switching to continuous batching, adding speculative decoding with a 1B draft model, and switching precision to INT4 via AWQ.

(a) Explain precisely *why* static batching produces the low GPU utilization number reported (24%), referencing what happens to short sequences in the batch. **[0.5 marks]**

(b) The team enables **continuous batching only**. Taking the **lower bound** of the industry-typical 70–85% utilization range, compute the **expected new throughput** (requests/second), assuming throughput scales linearly with GPU utilization from the 24% baseline. **[0.75 marks]**

(c) The team is deciding whether to *also* add speculative decoding on top of continuous batching. Code completion is one of the best-suited tasks for speculative decoding — **explain why**, contrasting it with a task type where speculative decoding would perform poorly, and explain mechanistically why the *verification* step does not require sequential passes even though normal generation does. **[0.75 marks]**

(d) Separately, the team evaluates switching the 70B model from fp16 to INT4 via AWQ. State the **memory reduction factor**, and identify the **one architectural benefit** this unlocks regarding GPU count that is *distinct* from "it uses less memory." **[0.75 marks]**

(e) The team can only ship **one** of the three changes this sprint. **Rank the three changes in the order you would implement them**, and justify with respect to (i) implementation risk, (ii) expected throughput gain, and (iii) quality risk. **[0.75 marks]**

Expected Answer

(a) In static batching, all sequences must wait for the **longest** sequence before the batch slot is reused. Short sequences that finish early sit idle — reserved but unused — while the GPU keeps processing the longest sequence. This idle time across most sequences drives utilization down to the 20–30% range.

(b) Baseline: 24%→5 req/s. New throughput $= 5 \times \frac{70}{24} \approx \mathbf{20.4}$ req/s (accept 20–21).

(c) Speculative decoding works best when the draft model's proposed tokens are frequently **correct** — code completion has strong local syntactic/idiomatic patterns (variable names, boilerplate), so the small model agrees often and few tokens are rejected. Creative writing has a much wider space of acceptable continuations, so the draft diverges quickly, causing frequent rejection and little/no speedup. Mechanistically: normal generation needs token t_k as *input* to compute t_{k+1} (hard sequential dependency), but during verification the draft tokens are already known, so the large model uses teacher-forcing to compute $P(t_i \mid t_1, \ldots, t_{i-1})$ for all K positions **simultaneously** in one batched forward pass, exactly like training.

(d) fp16 (2 bytes)→INT4 (0.5 bytes) is a $4\times$ reduction. At fp16, 70B needs ∼140 GB (needs ≥2 GPUs just for weights); at INT4, ∼35 GB **fits on a single 40/80 GB A100** — changing the deployment topology from mandatory multi-GPU tensor parallelism to single-GPU serving (no cross-GPU communication overhead), not merely "using less RAM."

(e) Representative ranking (grade on justification, not order): **1. Continuous batching** — lowest implementation risk (no model changes, zero quality risk, pure scheduler change), 3–4× throughput per the course data. **2. INT4/AWQ** — moderate risk (needs validation), nonzero quality risk (3–7% perplexity increase) but large gain once validated. **3. Speculative decoding** — needs a second draft model and orchestration; real but smallest structural benefit (2–3×) for the most added complexity, best deferred.

Criteria	Marks
(a) Correct static-batching idle-time explanation	0.5
(b) Correct linear-scaling throughput calculation	0.75
(c) Code-completion vs. poor task + mechanistic verification reasoning	0.75
(d) Correct $4\times$ factor and single-GPU topology benefit	0.75
(e) All three axes justified for each of the three changes	0.75

Q3 Descriptive | Feature Store Failure Diagnosis | 2.5 marks

A churn-prediction pipeline computes a feature — "user's average session length over the trailing 14 days" — using a nightly Spark job that writes into an offline Parquet store, then syncs into a Redis online store for serving.

(a) The offline training pipeline queries this feature using **all data up to the current date** at the time each training row is constructed, rather than restricting to data available strictly before each row's label date. Name this failure precisely and state the mechanism by which it inflates offline evaluation metrics. **[0.5 marks]**

(b) A colleague proposes fixing the bug by recomputing the feature **live from the production database on every serving request** rather than relying on the Redis online store. Give **two distinct, concrete** problems this introduces at serving time. **[0.5 marks]**

(c) Three months after the point-in-time bug is fixed and the model is redeployed, live precision drops sharply over two weeks while p99 latency and error rate remain completely flat. Feature drift alerts fire on two input features. Name the **most likely first thing to check** before concluding this is genuine distribution shift, and explain why that check should come first. **[0.5 marks]**

(d) Assume the check in (c) rules out a pipeline bug, and the drift is confirmed statistically. Classify this scenario as **covariate shift, label shift, or concept drift**, and justify using $P(X)$ / $P(Y \mid X)$ / $P(Y)$ notation. **[0.5 marks]**

(e) Given this is a churn model (not fraud or news-ranking), which **retraining trigger** — scheduled, drift-triggered, or performance-triggered — is most appropriate as the *primary* trigger, and what is the main practical risk of relying on it as your *only* trigger? **[0.5 marks]**

Expected Answer

(a) This is a **violation of point-in-time correctness** (future information leakage). Computing the feature with post-label-date data implicitly previews information unavailable in production; the model exploits future correlations that won't exist at inference time, so offline metrics look artificially strong and don't transfer.

(b) Any two: **Added latency** (a live DB query adds tens–hundreds of ms vs. sub-ms Redis lookup, risking SLA breach); **Fragility/load** (OLTP database wasn't designed for high-throughput analytical aggregation at serving scale); **Reintroduced training-serving skew** (if live query logic diverges even slightly from the offline definition).

(c) Check first whether this is a data pipeline bug, not genuine drift — per the course, ∼80% of apparent drift alerts trace back to pipeline bugs (schema changes, null handling, wrong date windows) rather than real shift. It's checked first because it is both the most common cause and cheapest to fix; skipping it risks an expensive retrain on the same broken data.

(d) This is **covariate shift**: $P(X)$ has changed while the underlying relationship $P(Y \mid X)$ is assumed unchanged. Distinguished from concept drift ($P(Y \mid X)$ itself changes) and label shift ($P(Y)$ changes while $P(X \mid Y)$ stays fixed).

(e) Scheduled retraining (e.g., monthly) — churn dynamics evolve over weeks/months, and ground-truth labels require a waiting period, making performance-triggered retraining slow to react. Main risk: it won't catch a sudden fast-moving shift (pricing change, outage) between cycles, so degradation could persist undetected for the interval length.

Criteria	Marks
(a) Correct terminology + leakage mechanism	0.5
(b) Two distinct, concrete serving-time problems	0.5
(c) Identifies pipeline-bug check with correct reasoning	0.5
(d) Correct classification with $P(X)/P(Y \mid X)$ justification	0.5
(e) Correct primary trigger + practical risk	0.5

Q4 Descriptive | Problem Framing Under Constraints | 2.5 marks

A ride-hailing company wants to reduce cancelled rides. A junior data scientist proposes: "predict, for each driver-rider pair at match time, which of our 12 driver incentive types (bonus, surge-boost, streak-bonus, etc.) will most reduce cancellation risk, and apply it."

(a) Is "reduce cancelled rides," as originally stated, an ML problem? Justify using the three-part definition of an ML problem from the course. **[0.5 marks]**

(b) The junior data scientist's framing treats this as a **12-class multiclass classification** problem. State **one concrete operational cost** of this framing that would appear the day the company launches a **13th** incentive type, and propose an alternative framing (name the task type) that avoids this cost. **[0.5 marks]**

(c) A senior engineer suggests decoupling into two separately trained models: a **cancellation-risk model** and an **incentive-effectiveness model**. Give **one argument for** and **one argument against** this decoupling, specific to this ride-hailing scenario. **[0.5 marks]**

(d) The stakeholder wants the **ML metric** to have a demonstrable, traceable link to a **business metric**. Propose one specific business metric and one specific ML metric, and state the direction-of-movement assumption you would need to validate empirically. **[0.5 marks]**

(e) Six weeks after launch, the cancellation-risk model shows strong offline AUC, but the overall cancellation rate hasn't moved. Give **two distinct hypotheses** (other than "the model is bad") for why a good model might fail to move the business metric. **[0.5 marks]**

Expected Answer

(a) No — this is a **business problem**, not an ML problem: it doesn't yet specify a well-defined **input**, **output**, and **objective function**. It must first be framed (e.g., as a classification or ranking task with concrete inputs/outputs) before it becomes solvable by ML.

(b) Operational cost: with 12 fixed output classes, a 13th incentive type is **not representable** by the existing output space — the model must be architecturally modified and **fully retrained** before it can ever recommend it. Alternative: a **regression/scoring** task — score each (pair, incentive type) combination and rank/select the top score at serving time, so a new type can be scored immediately without retraining.

(c) *For:* the two models can be updated/evaluated/rolled back **independently** (cancellation-risk may need frequent seasonal retraining; incentive-effectiveness only when new programs launch). *Against:* the effects may be **causally entangled** — a rider's baseline risk may interact with *which* incentive is most effective, and reconstructing this from two independent models' outputs may lose signal a joint model would capture.

(d) Business metric: **overall ride cancellation rate**. ML metric: **predicted cancellation probability** (or classifier AUC on held-out matches). Assumption to validate: that reducing the *model's predicted* probability via the recommended incentive causes a **corresponding real-world reduction** in actual cancellations — the proxy's improvement must be verified to track the business metric, not simply assumed.

(e) Any two: (1) **Output never acted upon operationally** — recommendation computed but not consistently surfaced, or the intervention pipeline had a bug. (2) **Threshold makes the intervention economically ineffective** — incentive cost exceeds the value of rides saved at the chosen operating point. (3) **Intervention itself is ineffective** — at-risk riders correctly identified, but the chosen incentive types don't change behavior for this population.

Criteria	Marks
(a) Correct No + three-part ML-problem justification	0.5
(b) Concrete operational cost + valid alternative framing	0.5
(c) Scenario-specific argument for and against	0.5
(d) Valid business/ML metric pair + direction-of-movement assumption	0.5
(e) Two distinct, plausible hypotheses	0.5

Section B — Multiple Options Correct MCQ (2 marks)

Each question carries 0.5 marks. Full mark awarded only when all correct options are selected and no incorrect options are selected. No negative marking. Select ALL correct options.

Q5 Multiple Correct | Feature Stores & Point-in-Time Correctness | 0.5 mark

Which of the following statements about feature stores and point-in-time correctness are TRUE?

A) **An online feature store is typically backed by a low-latency key-value store so that per-entity feature lookups return within a sub-millisecond serving budget ✓**
B) Point-in-time joins exist mainly to compress the offline Parquet store so that nightly batch materialization jobs finish within their scheduled window
C) **Computing a feature from all data up to today, rather than only data available before each label date, leaks future information into the training set ✓**
D) An offline feature store and an online feature store must run different feature computation logic, because sharing one definition across both makes serving too slow to meet its latency target

Answer: A, C

Q6 Multiple Correct | Latency, Throughput & SLOs | 0.5 mark

Which of the following statements about latency, throughput, and SLOs are TRUE?

A) **The p99 latency of a service can be several times worse than its mean latency, even when the large majority of requests are still served quickly ✓**
B) **Raising the batch size can increase overall system throughput even when it also raises the latency seen by each individual request ✓**
C) Under continuous batching, system throughput is still exactly the reciprocal of mean per-request latency, so measuring either one alone fully determines the other
D) **An SLA written only against mean latency can completely hide a severe tail-latency problem that degrades the experience of a small fraction of users ✓**

Answer: A, B, D

Q7 Multiple Correct | Technical Debt in ML Systems (Sculley et al.) | 0.5 mark

Which of the following statements about technical debt in ML systems are TRUE?

A) A "pipeline jungle" arises mainly when teams add encryption to data pipelines without a matching key-rotation policy, so old keys accumulate over time
B) **Glue code around a general-purpose model makes it hard to swap that model out later, because significant surrounding integration logic must be rewritten ✓**
C) Correction cascades reliably surface in standard model accuracy metrics well before they cause any downstream damage, so accuracy tracking alone catches them early
D) **Undeclared consumers that silently read a pipeline's output are a recognised contributor to pipeline jungles and make changes risky ✓**

Answer: B, D

Q8 Multiple Correct | CI/CD and Monitoring for ML Systems | 0.5 mark

Which of the following statements about CI/CD and monitoring for ML systems are TRUE?

A) **A monitoring-vendor executive observed that the large majority of drift alerts seen in practice trace back to pipeline bugs rather than to genuine distribution shift ✓**
B) A CI/CD performance gate should auto-promote any candidate model that beats production by any positive margin on the held-out set, since a nonzero improvement is still a real improvement
C) **Covariate shift denotes a change in the input distribution $P(X)$ while the conditional relationship $P(Y \mid X)$ is assumed to stay stable ✓**
D) **Data-validation checks such as schema, null-rate and volume serve a fundamentally different purpose from a model performance gate in the pipeline ✓**

Answer: A, C, D

Section C — Match the Following (1 mark)

No credit is given for guessing. Several Column A items map to more than one Column B item. Each Column B item may be used once, more than once, or not at all. Some Column B items are unused decoys.

Q9 Match the Following | Serving, Storage & ML System Failure Modes | 1 mark

Match each item in **Column A** to *all* item(s) it corresponds to in **Column B**. Several Column A items map to more than one Column B item; two Column B items are unused decoys; and one decoy is deliberately a near-miss for a Column A item it does *not* match. Each Column B item may be used once, more than once, or not at all. **Write your answer as a single Python `dict` of lists**, keyed by the Column A number (`int`) with a sorted list of Column B letters (`str`) as the value — e.g. `{1: ["B", "D"], 2: ["A"], 3: []}`. Include every Column A item as a key; use an empty list if it matches nothing.

Column A

1. Training-serving skew
2. OLTP database
3. Shadow deployment
4. Canary deployment
5. Correction cascade
6. PagedAttention
7. Continuous batching

Column B

A. Optimized for high-concurrency, low-latency single-entity point reads and writes

B. Candidate model runs on real inputs but its predictions are never served to users

C. Model B is added to patch Model A's failure case, and later itself fails on a case created by that patch

D. Root cause is preprocessing logic that diverges between the training and the serving code paths

E. Candidate model serves a small, deliberately capped fraction of live production traffic

F. Optimized for wide aggregate scans and column-oriented reads over millions of historical rows

G. Mitigated by raising replica count behind a load balancer, without changing the model

H. Fixed by persisting the full preprocessing pipeline inside the model artefact, not just the weights

I. Reduces KV-cache memory waste by allocating it in fixed-size blocks instead of one contiguous span

J. Raises GPU utilization by adding and evicting sequences from the running batch at each decode step

K. Can be detected early by comparing feature distributions computed offline against those logged at serving time

L. Provides a safe rollback path because only a bounded blast radius of users is exposed to a bad release

Answer: `{1: ["D", "H", "K"], 2: ["A"], 3: ["B"], 4: ["E", "L"], 5: ["C"], 6: ["I"], 7: ["J"]}`

Decoys: F describes an OLAP store, not OLTP (item 2); G is a horizontal-scaling fix that matches none of the Column A items. Multi-maps: training-serving skew is both caused by (D) and detected via (K) and remedied by (H); canary deployment is defined by (E) and valued for (L).

Course: ML System Design and LLMOps **Year:** 4th Year **Instructor:** Priyansh Saxena